

INF672 / Unidad 1 / GL01

GUÍA DE LABORATORIO

Instalación de Anaconda y exploración de Jupyter Notebook

Poner en marcha el entorno de trabajo de la materia y familiarizarse con el cuaderno interactivo.

Machine Learning · 6º semestre · Gestión 2026-02

Duración: 3 h de laboratorio

UATF · Ing. Informática · Docente: M.Sc. Huáscar Fedor Gonzales Guzmán

NOTA Convenciones de esta guía

Bloques oscuros = comandos de terminal (Anaconda Prompt / Terminal). `In [ ] / Out[ ]` = celdas dentro de Jupyter Notebook.

01 Competencia y objetivos

Competencia de la práctica

Instala y configura un entorno de trabajo reproducible para Machine Learning basado en Anaconda, y opera con solvencia la interfaz de Jupyter Notebook, reconociendo sus tipos de celda y su modelo de ejecución.

Objetivos específicos

- Instalar Anaconda de forma local en el sistema operativo propio (Windows, Linux o macOS).
- Comprender qué es un entorno (environment) de conda y por qué es clave para la reproducibilidad.
- Crear un entorno propio para la materia e instalar en él librerías de terceros.
- Explorar Jupyter Notebook: tipos de celda, contadores `In[ ]/Out[ ]`, Markdown, kernel y exportación.
- Producir y entregar un cuaderno propio como evidencia de la práctica.

02 Requisitos y materiales

Requisito	Detalle
Equipo	PC/laptop con Windows 10/11, Linux (Ubuntu/Debian u otra) o macOS.
Espacio en disco	Al menos 5 GB libres para la instalación completa de Anaconda.
Memoria RAM	Mínimo 4 GB; recomendado 8 GB para trabajar cómodamente.

Requisito	Detalle
Conexión a Internet	Necesaria para la descarga del instalador y de las librerías.

03 Marco conceptual

Un proyecto de Machine Learning no vive solo de Python: necesita un conjunto de librerías numéricas y de visualización trabajando en versiones compatibles entre sí. Instalarlas una por una y mantenerlas sincronizadas es tedioso y frágil. Anaconda resuelve exactamente ese problema.

DEFINICIÓN Anaconda

Anaconda es una distribución de Python (y R) orientada a ciencia de datos que empaqueta, en una sola instalación, el intérprete de Python, cientos de librerías científicas ya probadas juntas (Numpy, Pandas, Matplotlib, Scikit-learn...), la aplicación Anaconda Navigator y el gestor de paquetes y entornos conda.

DEFINICIÓN conda

conda es el gestor de paquetes y de entornos de Anaconda. A diferencia de pip, además de instalar librerías de Python puede instalar dependencias de más bajo nivel (compiladores, librerías de C) y administrar entornos aislados. Es la herramienta que usaremos para crear nuestro entorno de la materia.

NOTA

Existe también Miniconda, una versión mínima que trae solo Python + conda. En esta guía usamos Anaconda completo porque incluye Navigator y el stack científico listo para usar, lo que facilita el arranque.

04 Bloque A — Instalación de Anaconda

4.1 Descarga del instalador

Ingresa al sitio oficial y descarga el instalador de Anaconda Distribution correspondiente a tu sistema operativo. Descarga siempre desde la fuente oficial para evitar instaladores modificados.

> SITIO OFICIAL DE DESCARGA

Página oficial (elige tu sistema operativo):

<https://www.anaconda.com/download>

□ CAPTURA

Página de descarga de Anaconda con las opciones Windows / macOS / Linux.

Windows

1. Ejecuta el archivo .exe descargado (por ejemplo Anaconda3-2024.xx-Windows-x86_64.exe).
2. Acepta la licencia y elige la instalación "Just Me" (solo para tu usuario).

3. Deja la ruta por defecto. En la pantalla de opciones avanzadas mantén marcado "Register Anaconda as my default Python".
4. Finaliza la instalación y abre el menú Inicio: busca "Anaconda Prompt".

ADVERTENCIA

En Windows, NO marques la opción "Add Anaconda to my PATH environment variable" (aparece resaltada como no recomendada). Deja esa casilla desmarcada y trabaja siempre desde el Anaconda Prompt; agregarla al PATH suele romper otras instalaciones de Python del sistema.

Linux

El instalador de Linux es un script .sh que se ejecuta desde la terminal:

```
> TERMINAL - LINUX

# Ubícate donde descargaste el instalador y ejecútalo:
$ bash ~/Descargas/Anaconda3-2024.xx-Linux-x86_64.sh

# Acepta la licencia, confirma la ruta (~/.anaconda3) y responde 'yes'
# cuando pregunte si quieres inicializar conda.
```

macOS

En macOS puedes usar el instalador gráfico (.pkg) o el script de línea de comandos. Para el gráfico, haz doble clic en el .pkg y sigue el asistente. Verifica que descargas la versión correcta según tu chip (Apple Silicon M1/M2/M3 o Intel).

```
> TERMINAL - MACOS (ALTERNATIVA POR SCRIPT)

$ bash ~/Downloads/Anaconda3-2024.xx-MacOSX-arm64.sh
```

4.2 Verificación de la instalación

Abre el Anaconda Prompt (Windows) o una terminal nueva (Linux/macOS) y comprueba que conda responde:

```
> ANACONDA PROMPT / TERMINAL

$ conda --version
conda 24.x.x

$ python --version
Python 3.1x.x
```

Verás un prefijo (base) al inicio de la línea: indica que estás dentro del entorno por defecto de Anaconda, llamado base. También puedes abrir Anaconda Navigator desde el menú de aplicaciones para confirmar la instalación de forma visual.

□ CAPTURA

Anaconda Navigator abierto, mostrando Jupyter Notebook entre las aplicaciones.

4.3 Entornos (environments)

DEFINICIÓN Entorno (environment)

Un entorno es una instalación aislada de Python con su propio conjunto de librerías y versiones, independiente del resto del sistema. Cada proyecto puede tener su entorno, de modo que las versiones de un proyecto no interfieran con las de otro.

¿Por qué importa en Machine Learning? Porque un modelo entrenado con una versión de Scikit-learn puede comportarse distinto con otra. Aislar cada proyecto en su entorno hace el trabajo reproducible: cualquiera puede recrear el mismo entorno y obtener los mismos resultados. La regla práctica es: un entorno por proyecto, y nunca instales todo en base.

Crear el entorno desde Anaconda Navigator

Como instalamos Anaconda completo, crearemos el entorno de la materia con la interfaz gráfica de Navigator, sin escribir comandos:

1. Abre Anaconda Navigator y entra a la pestaña Environments, en la columna izquierda.
2. Pulsa el botón Create, en la parte inferior de la lista de entornos.
3. En el cuadro de diálogo, escribe el nombre del entorno: inf672.
4. Marca la casilla Python y elige la versión 3.14.6 en el desplegable.
5. Pulsa Create y espera unos segundos: el entorno inf672 aparecerá en la lista de entornos.

□ CAPTURA

Pestaña Environments de Navigator con el botón Create y el diálogo de nuevo entorno (nombre inf672, Python 3.14.6).

NOTA

Al seleccionar inf672 en la lista, el panel de la derecha muestra los paquetes instalados en ese entorno. Recién creado solo trae Python y sus dependencias básicas; en el siguiente paso le agregaremos las librerías de la materia.

Equivalente por terminal (opcional)

Si prefieres la línea de comandos, estas son las operaciones equivalentes sobre entornos:

> ANACONDA PROMPT / TERMINAL

```
# Crear el entorno inf672 con Python 3.14.6
$ conda create --name inf672 python=3.14.6

# Activarlo (el prefijo cambia de (base) a (inf672))
$ conda activate inf672

# Listar todos los entornos existentes
$ conda env list

# Salir del entorno actual
```

```
$ conda deactivate

# Eliminar un entorno por completo
$ conda remove --name inf672 --all
```

4.4 Instalación de librerías de terceros

Al entorno inf672 recién creado le agregaremos el stack básico de la materia. Puedes hacerlo desde Navigator o desde la terminal; antes conviene entender los dos gestores de paquetes:

	conda install	pip install
Origen	Canales de conda (defaults, conda-forge).	Índice PyPI.
Gestiona	Paquetes Python y dependencias de sistema.	Solo paquetes Python.
Cuándo usar	Preferente dentro de un entorno conda.	Cuando el paquete no está en conda.

ADVERTENCIA No mezcles conda y pip a la ligera

Dentro de un mismo entorno, instala primero todo lo posible con conda y recurre a pip solo al final, para lo que no exista en los canales de conda. Alternar ambos de forma desordenada puede dejar el entorno en un estado inconsistente y difícil de reparar.

Opción A — desde Anaconda Navigator

1. En la pestaña Environments, selecciona el entorno inf672.
2. En el panel de la derecha, cambia el filtro de "Installed" a "All".
3. Busca cada paquete por nombre (numpy, pandas, matplotlib, seaborn, scikit-learn, jupyter, notebook) y marca su casilla.
4. Pulsa Apply abajo a la derecha y confirma; Navigator resolverá las dependencias e instalará todo en el entorno inf672.

□ CAPTURA

Panel de paquetes de Navigator con el filtro en All y varios paquetes de la materia marcados para instalar.

Opción B — desde la terminal (más rápida para varios paquetes)

Activa el entorno e instala todo el stack en un solo comando usando el canal conda-forge, el más completo y actualizado de la comunidad:

```
> ANACONDA PROMPT / TERMINAL

$ conda activate inf672

# Stack numérico, de datos, visualización y ML + Jupyter
(inf672) $ conda install -c conda-forge numpy pandas matplotlib \
                                         seaborn scikit-learn \
                                         jupyter notebook -y
```

```
# Verificar que quedaron instaladas
(inf672) $ conda list numpy
```

NOTA Ejemplo con pip

Si más adelante necesitas un paquete que no esté en conda-forge, dentro del entorno activo harías: `pip install nombre-del-paquete`. pip se instala dentro del entorno y solo afecta a ese entorno.

4.5 Lanzar Jupyter Notebook

Con el entorno inf672 activo, hay dos formas de abrir Jupyter Notebook:

Opción A — desde la terminal

```
> TERMINAL — ENTORNO (INF672) ACTIVO

(inf672) $ jupyter notebook

# Se abre el navegador en http://localhost:8888
# La terminal queda 'ocupada' sirviendo el notebook: no la cierras.
```

Opción B — desde Anaconda Navigator

Abre Anaconda Navigator, selecciona arriba tu entorno inf672 en el desplegable “Applications on”, y pulsa Launch bajo Jupyter Notebook.

□ CAPTURA

Anaconda Navigator con el entorno inf672 seleccionado y el botón Launch de Jupyter.

ADVERTENCIA

Verifica siempre que lanzas Jupyter con el entorno inf672 activo. Si lo abres desde (base), no encontrará las librerías que instalaste en inf672 y obtendrás errores de `ModuleNotFoundError`.

05 Bloque B — Exploración de Jupyter Notebook

Ya con Jupyter abierto, crea un cuaderno nuevo: New ▶ Notebook ▶ Python 3 (ipykernel). Se abrirá un documento .ipynb con una celda vacía. Sobre este cuaderno haremos el recorrido.

5.1 La interfaz

- **Barra de menú y de herramientas:** guardar, agregar/mover celdas, ejecutar y cambiar el tipo de celda.
- **Área de celdas:** el cuerpo del cuaderno, donde escribes código o texto.
- **Indicador de kernel:** arriba a la derecha; un círculo lleno indica que el kernel está ocupado ejecutando.

5.2 Tipos de celda y modos

Una celda puede ser de tres tipos: Código (Python que se ejecuta), Markdown (texto con formato) y Raw (texto sin procesar). Cambias el tipo desde el menú o con atajos.

DEFINICIÓN Modo comando vs. modo edición

En modo edición (borde verde) escribes dentro de la celda. En modo comando (borde azul) operas sobre las celdas: crear, borrar, mover, cambiar tipo. Pulsa Enter para entrar a editar y Esc para volver a modo comando.

5.3 Ejecución: In[] y Out[]

Escribe tu primera celda de código y ejecútala con Shift + Enter (ejecuta y pasa a la siguiente). El número entre corchetes es el contador de ejecución:

In [1]:

```
mensaje = "Hola, Machine Learning"
print(mensaje)
```

Out[]:

```
Hola, Machine Learning
```

Cuando una celda devuelve un valor (no un print), Jupyter lo muestra como una celda Out[] con el mismo número:

In [2]:

```
2 ** 10
```

Out[2]:

```
1024
```

ADVERTENCIA El orden de ejecución importa

El número In[] indica el orden en que ejecutaste las celdas, no su posición en la página. Si ejecutas celdas salteadas, puedes usar una variable antes de definirla y obtener resultados confusos. Ante la duda: Kernel ▶ Restart & Run All para ejecutar todo de arriba hacia abajo en orden limpio.

5.4 Celdas Markdown y fórmulas

Convierte una celda a Markdown (Esc, luego M) para escribir texto con formato: títulos con #, listas con -, negritas con **texto**. Jupyter también renderiza fórmulas LaTeX entre signos \$, muy útil en ML.

EJEMPLO Fórmula de la regresión lineal en Markdown

Escribiendo en una celda Markdown: $\hat{y} = w_0 + w_1 x_1 + \dots + w_n x_n$

Jupyter lo renderiza como la ecuación de la predicción lineal, con subíndices y símbolos matemáticos formateados.

5.5 Primeros ejemplos con el stack

Comprobemos que las librerías instaladas funcionan dentro del cuaderno. Primero, Numpy y Pandas:

In [3]:

```
import numpy as np
import pandas as pd

notas = np.array([51, 68, 74, 89, 95])
print("Promedio:", notas.mean())
```

Out[]:

```
Promedio: 75.4
```

In [4]:

```
df = pd.DataFrame({
    "estudiante": ["Ana", "Beto", "Carla"],
    "nota": [89, 74, 95],
})
df
```

Out[4]:

	estudiante	nota
0	Ana	89
1	Beto	74
2	Carla	95

Y un primer gráfico con Matplotlib, que se dibuja en línea dentro del cuaderno:

In [5]:

```
import matplotlib.pyplot as plt

plt.plot([1, 2, 3, 4], [1, 4, 9, 16], marker="o")
plt.title("y = x²")
plt.show()
```

NOTA

Debajo de esa celda aparecerá la figura renderizada. Si no se muestra, ejecuta primero `%matplotlib inline` en una celda; en versiones recientes de Jupyter suele activarse por defecto.

5.6 Kernel y atajos

El kernel es el proceso de Python que ejecuta tus celdas y mantiene el estado (las variables definidas). Operaciones útiles desde el menú Kernel:

- **Interrupt:** detiene una ejecución colgada.
- **Restart:** reinicia el kernel; se pierden las variables en memoria.
- **Restart & Run All:** reinicia y ejecuta todo en orden; ideal para validar el cuaderno antes de entregar.

Atajos frecuentes (en modo comando, tras pulsar Esc)

Atajo	Acción
<code>`Shift + Enter`</code>	Ejecutar la celda y pasar a la siguiente.
<code>`Ctrl + Enter`</code>	Ejecutar la celda y quedarse en ella.
<code>`A` / `B`</code>	Insertar celda arriba (Above) / abajo (Below).
<code>`M` / `Y`</code>	Convertir a Markdown / a Código.
<code>`D D`</code>	Borrar la celda (pulsar D dos veces).
<code>`Z`</code>	Deshacer el borrado de una celda.

5.7 Guardado y exportación

El cuaderno se guarda como archivo `.ipynb` (Ctrl + S). Jupyter crea checkpoints automáticos que permiten revertir cambios. Para compartirlo en otros formatos usa File ► Download as:

- `.ipynb` — el cuaderno completo (código, texto y resultados). Es el formato de entrega.
- `.html` — versión de solo lectura para ver en el navegador sin Jupyter.
- `.py` — solo el código, como script de Python.

06 Entregable

Elabora un cuaderno propio que evidencie la práctica y súbelo a la plataforma virtual de la UATF.

EJEMPLO Qué entregar

Archivo: GL01_Apellido_Nombre.ipynb

1. Una celda Markdown inicial con tus datos (nombre, materia, fecha) y un título con formato.
2. Una celda Markdown que incluya al menos una fórmula LaTeX (por ejemplo, la de la regresión lineal).
3. Tres a cuatro celdas de código ejecutadas: un print, un cálculo con Numpy, un DataFrame de Pandas y un gráfico de Matplotlib.
4. Una captura de pantalla de la terminal mostrando la salida de `conda env list` con tu entorno `inf672` (insértala en una celda Markdown o entrégala aparte).
5. El cuaderno debe pasar limpio un Restart & Run All (sin errores) antes de entregar.

07 Rúbrica de evaluación

Criterio	Descripción	Pts.
Entorno creado	Evidencia del entorno <code>inf672</code> con las librerías instaladas (captura de <code>conda env list</code> / <code>conda list</code>).	25
Celdas Markdown	Título, datos personales y fórmula LaTeX correctamente renderizada.	20

Criterio	Descripción	Pts.
Celdas de código	Numpy, Pandas y Matplotlib ejecutados con salida coherente.	30
Ejecución limpia	El cuaderno corre sin errores con Restart & Run All; orden de celdas correcto.	15
Orden y presentación	Nombre de archivo correcto, cuaderno legible y bien organizado.	10

Total: 100 puntos

08 Referencias

- Raschka, S. & Mirjalili, V. — Python Machine Learning (bibliografía oficial de la asignatura).
- Documentación oficial de conda: <https://docs.conda.io>
- Documentación de Anaconda Distribution: <https://docs.anaconda.com>
- Documentación de Jupyter: <https://docs.jupyter.org>